

AMC .amc Execution Specification — Appendices

AMC .amc Execution Specification — Appendices

Implementation-level companion to the .amc Execution Specification. These appendices give, in typed language-agnostic pseudocode, the exact bodies that the Execution Spec delegated: (A) the built-in function library, (B) the instrument self-expansion algorithms, and (C) the day-count / schedule-roll conventions. An implementer (LLM or human) targeting C#, C++, Java, Rust, or similar should be able to transcribe each entry directly.

Pseudocode conventions

All values are **per-path vectors** of fixed length N . The pseudocode uses a single vector type `Vec` (an N -length array of `double`) and these primitives, each understood **element-wise with scalar broadcasting** (a scalar operand is treated as a constant `Vec`):

```
Vec          : an N-length array of double
scalar       : a double (broadcasts against any Vec)
Mat          : an (rows × cols) matrix of double

# element-wise vector ops (all return Vec)
a + b, a - b, a * b, a / b, a ** p      # arithmetic
max(a, b), min(a, b)                   # element-wise (binary)
abs(a), sqrt(a), exp(a), log(a)        # element-wise unary
a < b, a <= b, a > b, a >= b, a == b, a != b # → 0/1 mask Vec
a & b, a | b, ~a                        # element-wise mask AND/OR/NOT
select(mask, a, b)                     # mask ? a : b element-wise (BOTH a,b evaluated)
clip(x, lo, hi)                        # element-wise min(max(x,lo),hi)
full(N, c)                             # constant Vec of value c
norm_cdf(x)                            # standard-normal CDF, element-wise
```

Reductions over a *stack* of n vectors (a basket) operate along the stack axis:

```
stack(v_1, ..., v_n)                  : Mat of shape (n × N)
reduce_max(stack)                     : Vec (element-wise max down the stack axis)
reduce_min, reduce_sum                : likewise
sort_along_stack(stack)               : ascending sort along the n-axis, per path
```

Per-path **state** is a mutable map `state : label → Vec`. A built-in that takes a string `label` first argument **self-seeds**: on the first call for that label it creates `state[label]` from an `init` value, then updates it. The string label is matched **exactly** (case-sensitive).

The **market object** `md` and the per-date snapshot `snap` provide the queries of Execution-Spec §2. `t` is the current event's date (a scalar year-fraction).

Where the reference uses a specific constant or tie-break, it is stated inline and is **load-bearing** — changing it changes the numbers.

Appendix A — Built-in function library

The library has ~338 names, but most are **aliases** or **lowercase duplicates** of a smaller set of distinct functions. Each entry below gives the canonical name, its aliases, the signature, and the body. Functions are grouped by family; families A.1–A.4 need no market access (pure on their `Vec` arguments), A.5–A.9 touch state, the market, rates, or credit.

A.1 Arithmetic & shaping

All element-wise; arguments and results are `Vec`.

```
MAX(a, b, ...)      = reduce_max(stack(a, b, ...))      # variadic; also MAX([list])
MIN(a, b, ...)      = reduce_min(stack(a, b, ...))
ABS(x)              = abs(x)
SQRT(x)             = sqrt(x)
EXP(x)              = exp(x)
LOG(x) [alias LN]   = log(x)                          # natural log
LOG10(x)            = log(x) / log(10)
LOG2(x)             = log(x) / log(2)
LOG1P(x)            = log(1 + x)
EXPM1(x)            = exp(x) - 1
POW(x, p)           = x ** p                          # identical to the ** operator
SIGN(x)             = select(x > 0, 1, select(x < 0, -1, 0))
POS(x) [positive_part] = max(x, 0)
NEG(x) [negative_part] = max(-x, 0)
CEIL(x)             = element-wise ceiling
FLOOR(x)            = element-wise floor
ROUND(x[, ndigits]) = element-wise round half-to-even to ndigits (default 0)
MOD(a, b)           = a - b * floor(a / b)             # follows the % operator
CLAMP(x, lo, hi) [CLIP, COLLAR] = clip(x, lo, hi)
CAP_AT(x, c) [CAPPED_AT] = min(x, c)
FLOOR_AT(x, f) [FLOORED_AT] = max(x, f)
INDIC(cond) [INDICATOR] = select(cond != 0, 1, 0)
STEP(x)             = select(x > 0, 1, 0)
WHERE(cond, a, b)    = select(cond != 0, a, b)
```

Reduction helpers over a list/group (variadic or single-list argument):

```
SUM(a, ...)          = reduce_sum(stack(a, ...))
AVG(a, ...) [MEAN, AVERAGE] = SUM(a, ...) / n        # n = number of arguments
PRODUCT(a, ...)      = element-wise product down the stack
MEDIAN(a, ...)        = per-path median of the stack (type-7; even n: mean of two middle)
VAR(a, ...)           = population variance (ddof = 0) down the stack
STDEV(a, ...)         = sqrt(VAR(...))
RANGE(a, ...)         = reduce_max(stack) - reduce_min(stack)
RMS(a, ...)           = sqrt(AVG(a_i ** 2))
COUNT(a, ...)        = number of arguments where value != 0, per path
ANY(a, ...)           = select(reduce_max(stack(a!=0,...)) > 0, 1, 0)
ALL(a, ...)           = select(reduce_min(stack(a!=0,...)) > 0, 1, 0)
PERCENTILE(p, a, ...) / QUANTILE(q, a, ...) = type-7 linear-interpolated quantile down the stack
```

Order-statistic tie-breaks. MEDIAN, PERCENTILE, QUANTILE use the **type-7** (linear-interpolation) convention: sort ascending, index $h = (n - 1) \cdot q$, interpolate between

`floor(h)` and `ceil(h)`. This must match the engine and the Engine Spec's quantile convention exactly.

A.2 Option & payoff shapes

```
CALL_PAYOFF(S, K) [CALL] = max(S - K, 0)
PUT_PAYOFF(S, K) [PUT] = max(K - S, 0)
STRADDLE(S, K) = abs(S - K)
DIGITAL(x[, K]) [BINARY] = select(x >= K, 1, 0) # K defaults to 0
DOUBLE_DIGITAL(x, lo, hi) = select((x >= lo) & (x <= hi), 1, 0)
CALL_SPREAD(x, K, smoothing=None, reference=1.0)
    = SMOOTH_STEP(x, K, smoothing, "call_spread", reference) #
see A.3
COLLAR(x, lo, hi) = clip(x, lo, hi)
```

Closed-form European prices (used where a sub-payoff is itself an analytic option; all element-wise over per-path `S`, `K`, `sigma`):

```
NORM_CDF(x) = 0.5 * (1 + erf(x / sqrt(2)))

BS_EURO_CALL(S, K, r, q, T, sigma):
    if T <= 0: return max(S - K, 0)
    sig_T = sigma * sqrt(T)
    fwd = S * exp((r - q) * T)
    if all(sig_T <= 1e-12): return exp(-r*T) * max(fwd - K, 0)
    sig_T_safe = select(sig_T > 1e-12, sig_T, 1e-12)
    d1 = (log(S / K) + (r - q + 0.5*sigma*sigma)*T) / sig_T_safe
    d2 = d1 - sig_T_safe
    return S*exp(-q*T)*NORM_CDF(d1) - K*exp(-r*T)*NORM_CDF(d2)

BS_EURO_PUT(S, K, r, q, T, sigma):
    # by put-call parity, or symmetric formula with N(-d2), N(-d1)
    return BS_EURO_CALL(...) - S*exp(-q*T) + K*exp(-r*T)

BACHELIER_EURO_CALL(S, K, r, T, sigma): # normal model; for rates/spreads set r=0
    if T <= 0: return max(S - K, 0)
    F = S * exp(r * T)
    sig_T = sigma * sqrt(T)
    sig_T_safe = select(sig_T > 1e-12, sig_T, 1e-12)
    d = (F - K) / sig_T_safe
    phi = exp(-0.5*d*d) / sqrt(2*pi)
    return exp(-r*T) * ((F - K)*NORM_CDF(d) + sig_T*phi)

BACHELIER_EURO_PUT(S, K, r, T, sigma): # = BACHELIER_EURO_CALL + exp(-rT)*(K - F)
```

A.3 Smooth / differentiable shapes

Used for AAD-friendly approximations of digitals/barriers.

```
SIGMOID(x, scale=1.0) = 1 / (1 + exp(-x / scale)) # → STEP(x) as scale → 0

SOFT_MAX(beta, a, ...) [SOFTMAX]: # → MAX as beta → +∞
    s = stack(a, ...) # shape (n × N)
    return log( sum over stack of exp(beta * s) ) / beta

SOFT_MIN(beta, a, ...) [SOFTMIN]:
    return -SOFT_MAX(beta, -a, ...) # equivalently beta<0 in
SOFT_MAX
```

```
SMOOTH_STEP(x, K, smoothing=None, kind=None, reference=1.0) [SMOOTH_INDICATOR]:
# smooth approximation of 1[x >= K]. `smoothing` is a fraction of `reference`
# (typically S0); default ≈ 1% of reference. Three kinds:
# "call_spread" (default): clip( (x - (K - δ)) / (2δ), 0, 1 ), δ = smoothing *
reference
# "logistic":          SIGMOID( (x - K) / δ )
# "hard":              STEP(x - K)
δ = (smoothing if smoothing is not None else 0.01) * reference
if kind in (None, "call_spread"): return clip((x - (K - δ)) / (2*δ), 0, 1)
if kind == "logistic": return SIGMOID(x - K, δ)
if kind == "hard": return select(x >= K, 1, 0)
```

A.4 Rainbow / basket / order-statistic families

These reduce over a **basket** (a list/group of underlyings). Build a stack `S = stack(asset_1, ..., asset_n)` of shape `(n × N)`, then reduce per path.

```
BEST_OF(basket) [BEST, MAX of basket] = reduce_max(S) # per-path max across
assets
WORST_OF(basket) [WORST, MIN of basket] = reduce_min(S)
AVG_OF(basket) [BASKET_AVG] = reduce_sum(S) / n
SUM_OF(basket) = reduce_sum(S)

BEST_OF_IDX(basket) = argmax along the stack axis, per path (0-based asset index as a
Vec)
WORST_OF_IDX(basket) = argmin along the stack axis, per path
ARGMAX_OF, ARGMIN_OF = aliases of the above

KTH_BEST(k, a, ...): # k-th largest, 1-based
s = sort_along_stack(stack(a, ...)) # ascending, shape (n × N)
return s[n - k] # row (n-k) = k-th from the top
KTH_WORST(k, a, ...) = s[k - 1] # k-th from the bottom
SUM_BEST_K(k, a, ...) = reduce_sum of the top-k rows of the sorted stack
SUM_WORST_K(k, a, ...) = reduce_sum of the bottom-k rows

BASKET(weights, a, ...): # fixed-weight weighted sum
out = weights[0] * a_0
for i in 1..n-1: out = out + weights[i] * a_i
return out

ABSOLUTE_BASKET(weights, names...): # like BASKET but each component is first
# converted into the product currency via md.in_currency before weighting.
```

The *_EXCLUDING family (Himalaya / best-of-excluding-locked, etc.). These take a basket and an `exclude` argument (a scalar position, a per-path integer Vec of positions, or a list of such), plus an optional `min_keep`:

```
build INCLUDE-MASK M of shape (n × N):
M[:, :] = true
for each exclude entry e (in the order given):
for each path p:
pos = e (scalar) or e[p] (per-path); skip if pos < 0 (-1 sentinel = no-op)
# apply only if it keeps at least min_keep positions on this path:
if (count of true in M[:, p]) - 1 >= min_keep:
M[pos, p] = false
# min_keep default 0; with 0, an all-excluded path is an error.
# excludes are applied in order, so when the floor binds, EARLIER excludes win.

BEST_OF_EXCLUDING(basket, exclude, min_keep=0) = reduce_max( select(M, S, -inf) )
WORST_OF_EXCLUDING(basket, exclude, min_keep=0) = reduce_min( select(M, S, +inf) )
AVG_EXCLUDING(basket, exclude, min_keep=0) = sum(included)/count(included), per path
```

```

SUM_EXCLUDING                                = sum of included
MEDIAN_EXCLUDING                            = per-path median over included (masked→NaN, nan-median)
STDEV_EXCLUDING / VAR_EXCLUDING              = population (ddof=0) over included;
count=1 → 0
BEST_OF_IDX_EXCLUDING / WORST_OF_IDX_EXCLUDING = argmax/argmin over included positions

```

Consecutive-elimination (the Himalaya driver, fixed elimination count):

```

EXCLUDE_CONT_WORST(basket, keep_min) / EXCLUDE_CONT_BEST(basket, keep_min):
    # repeatedly remove the worst (resp. best) member until exactly keep_min remain.
    # Returns a per-path survivor MASK (n × N). The eliminated member may differ
    # per path, but the COUNT removed (n – keep_min) is fixed at compile time, so
    # this is a static unroll of (n – keep_min) vectorised argmin/argmax passes –
    # no data-dependent control flow.
    alive = all-true (N × n)
    repeat (n – keep_min) times:
        masked = select(alive, S, +inf for worst / -inf for best)
        j*      = argmin(masked) per path          # (argmax for BEST)
        alive[path, j*] = false
    return alive

```

A.5 Running accumulators (self-seeding per-path state)

Each takes a string `label` first argument, self-seeds `state[label]`, updates it, **writes it back into the readable environment** under `label`, and returns it. A single call inside an expanded `for/schedule` accumulates across iterations because every iteration reuses the same label. Init defaults are **load-bearing**:

```

RUNNING_SUM(label, x, init=0):
    if label not in state: state[label] = full(N, init)
    state[label] = state[label] + x
    return state[label]

RUNNING_MAX(label, x, init=-1e18) [SUP]:          # default seed -1e18
    if label not in state: state[label] = full(N, init)
    state[label] = max(state[label], x)
    return state[label]

RUNNING_MIN(label, x, init=+1e18) [INF]:          # default seed +1e18
    if label not in state: state[label] = full(N, init)
    state[label] = min(state[label], x)
    return state[label]

RUNNING_MEAN(label, x, init=0) [RUNNING_AVG]:     # online mean via hidden sum+count
    if (label__sum) not in state:
        state[label__sum] = full(N, init)
        state[label__count] = full(N, 0)
    state[label__sum] = state[label__sum] + x
    state[label__count] = state[label__count] + 1
    state[label] = state[label__sum] / max(state[label__count], 1)
    return state[label]

RUNNING_PROD(label, x, init=1): multiplicative analogue of RUNNING_SUM

```

Ratio / performance accumulators (the cliquet/snowball families). `RATIO(asset)` = `asset / anchor` (the inception anchor, §A.7); `PERF` = `RATIO – 1`. The `*_SUM`, `*_PROD`, and `_IF` variants accumulate these per call:

```

RATIO_SUM(asset, cap=None, floor=None):    running sum of clip(RATIO(asset), floor, cap)
PERF_SUM(asset, cap=None, floor=None):     running sum of clip(PERF(asset), floor, cap)

```

RATIO_PROD / PERF_PROD:	running product of (1 + clipped perf), etc.
*_SUM_IF(asset, cond, ...) / *_PROD_IF:	accumulate only on paths where cond != 0 (the condition is the SECOND argument)

SNAPSHOT stores a value for the rest of the path (the cum-div spot, an inception level, a captured rate):

```
SNAPSHOT(label, value):
    state[label] = value          # overwrite-store, persists along the path
    return value
```

A.6 Barrier survival (discrete and continuous monitoring)

BARRIER_SURVIVAL maintains a per-path survival latch in `[0,1]` keyed by `label`. Discrete monitoring multiplies by a per-observation no-hit factor; continuous monitoring adds a **Brownian-bridge continuity correction** between observations.

```
BARRIER_SURVIVAL(label, value, barrier, direction,
                  monitoring="discrete", sigma=None,
                  smoothing=None, kind=None, reference=1.0, init=1.0):
    up = (direction == "up")
    B = barrier
    if label not in state: state[label] = full(N, init)    # survival starts at init
    (1.0)

    if monitoring == "discrete":
        # per-observation hit probability via the smoothed indicator (A.3):
        if up: p_hit = SMOOTH_STEP(value, B, smoothing, kind, reference)    # 1
    above B
        else: p_hit = 1 - SMOOTH_STEP(value, B, smoothing, kind, reference)    # 1
    below B
        state[label] = state[label] * (1 - p_hit)
        return state[label]

    if monitoring == "continuous":
        # Brownian bridge between the previous observation (S_prev, t_prev) and now.
        # Work in log-space; variance of the bridge over the step is  $\sigma^2 \cdot \Delta t$ .
        if first call for label:
            state[label__prev_val] = value; state[label__prev_t] = t; return
    state[label]
        a = log(max(state[label__prev_val], 1e-12))    # log S_prev
        b = log(max(value, 1e-12))    # log S_now
        dt = max(t - state[label__prev_t], 1e-12)
        var = max(sigma*sigma*dt, 1e-12)    #  $\sigma^2 \cdot \Delta t$  (or an EWMA/realised
    estimate)
        logB = log(B)
        if up: p_no_hit = select(var>0, 1 - exp(-2*(logB - a)*(logB - b)/var), 0)
        else: p_no_hit = select(var>0, 1 - exp(-2*(a - logB)*(b - logB)/var), 0)
        state[label] = state[label] * p_no_hit
        state[label__prev_val] = value; state[label__prev_t] = t
        return state[label]
```

The bridge constant is load-bearing. The no-hit probability of a Brownian bridge from `a` to `b` over variance `var` not crossing level `logB` is `1 - exp(-2*(logB - a)*(logB - b) / var)` (up-barrier; symmetric for down). The factor **2** and the **product of the two endpoint gaps** are the exact continuity correction — an implementation that uses only the endpoint indicator (discrete) will misprice continuously-monitored barriers.

`CORRIDOR_SURVIVAL(label, value, lower, upper, ...)` is the two-sided analogue: multiply by the product of the up-bridge (at `upper`) and down-bridge (at `lower`) no-hit factors each step.

A.7 Dividend macros

These are **pre-substituted during lowering** (Execution-Spec §4.5–4.6), not evaluated as runtime functions. They appear here for completeness; the values C_i , Y_i , t_{ex_i} , t_{pay_i} are the bound dividend's constants and `Scum` is the hidden cum-div snapshot label captured at $t_{ex_i} - 10^{-9}$:

```
DIV_AMOUNT(eq)      → (Ci + Yi * Scum)          # lowered expression, path-dependent
DIV_C / DIV_C(eq) → Ci                          # scalar; call form rewrites to bare
token
DIV_Y / DIV_Y(eq) → Yi
DIV_T_EX / DIV_T_PAY → texi / tpayi
FWD_DIV_AMOUNT(eq) → Ci + Yi * md.forward(eq, obs_time, texi -) # compile-time
scalar
SUM_FWD_DIVS(eq, a, b) → Σ over ex-dates in [a,b] of (Ci + Yi * md.forward(eq,
0, texi -)) # scalar
SUM_REALIZED_DIVS(eq, a, b) → a hidden schedule_div over [a,b] + RUNNING_SUM(DIV_AMOUNT)
label
```

The spot-drop applied by the simulator at each ex-date is

$S(t_{ex^+}) = \max(S(t_{ex^-}) \cdot (1 - Y_i) - C_i, \text{spot_floor})$; the cum-div snapshot reads $S(t_{ex^-})$ so $\text{DIV_C} + \text{DIV_Y} \cdot \text{Scum}$ reconstructs `DIV_AMOUNT` exactly.

A.8 Rate built-ins (the single rate door)

All resolve through `md.bond_price / md.forward_rate` / a cash-account ratio. $B_{\text{ratio}}(ts, te)$ denotes the per-path money-market growth $B(te)/B(ts)$ (the `bank_account_ratio`; for deterministic rates it is the curve ratio, for stochastic rates the trapezoidal integral of the simulated short rate). t is the current event date.

```
DF(ccy, t_pay):
    if t_pay <= 0: return full(N, 1)
    return md.bond_price(ccy, t, t_pay) # per-path discount factor t → t_pay
                                         # = md.discount_factor

LIBOR(ccy, t_s, t_e, tenor=None, fixing="auto"):
    τ = t_e - t_s (must be > 0)
    if t_s < 0:
        # past period → observed fixing
        return full(N, md.past_fixing(index(ccy, "LIBOR", tenor), t_s))
    # in-arrears (or auto with t_s < t): realised compounded rate over [t_s, t_e]
    if fixing == "in_arrears" or (fixing == "auto" and t_s < t):
        return (B_ratio(t_s, t_e) - 1) / τ
    # in-advance (standard IBOR): forward simple rate seen at t for [t_s, t_e]
    # must be evaluated at t ≤ t_s; equals (P(t, t_s)/P(t, t_e) - 1)/τ
    return (md.bond_price(ccy, t, t_s) / md.bond_price(ccy, t, t_e) - 1) / τ

RFR(ccy, t_s, t_e, index=None):
    τ = (t_e - t_s) * 365/dc_denom # backward-looking risk-free rate
    # dc_denom = 360 for ACT/360 curves,
    else 365
    return (B_ratio(t_s, t_e) - 1) / τ

OIS(index, t_s, t_e):
    rate # compounded-in-arrears overnight
    past_compound = [] over past fixing days k in [t_s, min(t_e, 0)] of (1 + r_k * τ_k) #
    r_k observed
    future_compound = B_ratio(max(t_s, 0), t_e) # simulated forward compounding
    τ_total = day-count(t_s, t_e)
```



```

    return (past_compound * future_compound - 1) /  $\tau_{\text{total}}$ 

CMS(ccy, t_s, n, freq, tenor=None):
    # constant-maturity swap rate at t_s
    # the par swap rate of an n-period swap (period length `freq`) starting at t_s:
    # level (annuity)    A =  $\sum_{i=1..n} \text{freq} * P(t, t_s + i*freq)$ 
    # float leg value    F =  $P(t, t_s) - P(t, t_s + n*freq)$ 
    # swap rate          R = F / A
    # built from md.bond_price at t over the n+1 swap dates. Must be observed at
    # t ≤ t_s; the convention (in_advance) captures it at the swap start.
    A = sum over i in 1..n of freq * md.bond_price(ccy, t, t_s + i*freq)
    F = md.bond_price(ccy, t, t_s) - md.bond_price(ccy, t, t_s + n*freq)
    return F / A

YOY_INFLATION(index, t_s, t_e):
    # year-on-year inflation rate
    return md.forward(index, t, t_e) / md.forward(index, t, t_s) - 1

ZC_INFLATION_SWAP_RATE(index, T):
    # zero-coupon inflation swap fixed rate
    return (md.forward(index, t, T) / md.value(index, t)) ** (1/T) - 1

```

The rate door is the single coupling point. Every rate built-in is a closed-form combination of `bond_price`, a cash-account ratio, and (for past periods) observed fixings. Reproduce `bond_price/B_ratio` from the **same** simulated rate factor and trapezoidal $B(t)$ (Engine-Spec §3) and the rate built-ins match. The `fixing` convention (`in_advance` vs `in_arrears`) controls *which date the call must be evaluated at*; getting it wrong shifts the rate by one period.

A.9 Credit / survival built-ins

Resolve through the credit curve / hazard factor. `S_name(u)` is issuer `name`'s survival to time `u` (deterministic curve) or `exp(-∫ hazard)` (stochastic).

```

IS_ALIVE(issuer):
    return md.alive(issuer, t)
    # 0/1 alive indicator at t
    # = 1 if default_time > t, else 0

DEFAULTED(issuer) = 1 - IS_ALIVE(issuer)
JUST_DEFAULTED(issuer) = 1 on the path/step where default first occurs, else 0

SURVIVAL(issuer) = md.survival(issuer, t)
FWD_SURVIVAL(issuer, T) = md.forward_survival(issuer, t, T)
# S(t)
# determ: S(T)/S(t);
# stochastic:

conditional_survival(t,T,hazard_t)
HAZARD(issuer):
    # instantaneous hazard at t
    # deterministic curve: h = -(log S(t+dt) - log S(t)) / dt, dt = 1/365.25
    # stochastic: exp(hazard_factor_t)

RECOVERY(issuer) = md.recovery_rate(issuer)
# scalar R ∈ [0,1]

# Portfolio / basket credit:
BASKET_LOSS(names, weights=None):
    # fractional loss of a credit pool
    acc = 0
    for (nm, w) in zip(names, weights or all-ones):
        acc = acc + w * (1 - RECOVERY(nm)) * DEFAULTED(nm)
    return acc
    # weighted LGD-on-default sum

NUM_DEFAULTED(names) =  $\sum \text{DEFAULTED}(nm)$ 
# count; nth-to-default via >= n

FIRST_TO_DEFAULT(names) = select(NUM_DEFAULTED >= 1, 1, 0)

TRANCHE_LOSS(names, attach, detach, weights=None, notional=1):
    # tranche loss as a fraction of tranche notional:
    # L(t) = BASKET_LOSS(names, weights) /  $\sum \text{weights}$ 
    # TL(t) = clip((L(t) - attach)/(detach - attach), 0, 1)
    # portfolio loss fraction
    # 0 below a, 1 above d

```



```
L = BASKET_LOSS(names, weights) / sum(weights or ones)
return clip((L - attach) / (detach - attach), 0, 1)
```

Appendix B — Instrument self-expansion

Declared instruments (`bond`, `swap`, `forward`, `convertible`) **self-expand** during lowering into ordinary events. Below is the exact algorithm for each. All generated amounts are expression strings in the \$5 sub-language; all dates are year-fractions from `as_of`.

B.1 Coupon-date generation (shared)

Given `start`, `maturity`, and a frequency code (`A=1`, `S=0.5`, `Q=0.25`, `M=1/12`):

```
period = year_fraction(frequency)
n_cpns = round((maturity - start) / period)
cpn_dates = [ round(start + i*period, 10) for i in 1 .. n_cpns ]
```

The final coupon date coincides with `maturity` (within 10^{-9}). `start` defaults to `t0`; an `at <date> enter <inst>` statement overrides `start` to that date, shifting the whole timeline.

B.2 `bond` — risk-free and defaultable

```
bond NAME { notional, maturity, coupon, frequency, [issuer, recovery] }

cpn_amt = round(coupon * notional, 12)

# ---- risk-free bond (no issuer) ----
for T in cpn_dates:
    emit Payment(date=T, amount = cpn_amt)           # fixed coupon
emit Payment(date=maturity, amount = notional)       # redemption

# ---- defaultable bond (issuer given) : survival-weighted ----
# state: alive_prev seeded to 1 at t0.
for each cpn date T (k-th):
    emit Fixing (date=T, do = "alive_now = IS_ALIVE(issuer)") # 0/1 alive indicator
    emit Fixing (date=T, do = "just_defaulted = max(alive_prev - alive_now, 0)")
    emit Payment(date=T, amount = "alive_now * cpn_amt")      # coupon paid only if
alive
    emit Payment(date=T, amount = "just_defaulted * RECOVERY(issuer) * notional") #
recovery
    if T == maturity:
        emit Payment(date=T, amount = "alive_now * notional") # redemption if still
alive
    emit Fixing (date=T, do = "alive_prev = alive_now")       # roll the latch
```

The order within each date matters (the `(date, seq)` rule of §7.1): the alive indicator and `just_defaulted` are computed (lower `seq`) before the payments read them, and `alive_prev` is rolled last.

B.3 `swap` — fixed vs floating legs

A swap lowers to a single engine `SWAP(...)` call whose leg specifications are built from the block, **or** to explicit per-period coupon events. The leg model:

```
swap NAME { notional, maturity, frequency, fixed_rate, float=LIBOR|CMS|OIS, [legs] }
```

```

generate period boundaries  t_0 < t_1 < ... < t_n  from start, maturity, frequency
                                (with the chosen stub convention, Appendix C)
for each period [t_{i-1}, t_i]:
    τ_i = day_count(t_{i-1}, t_i)                                # accrual fraction
(Appendix C)
    # fixed leg coupon:
    fixed_cf_i = fixed_rate * τ_i * notional
    # float leg coupon (index resolved per `float`):
    #   LIBOR:  rate_i = LIBOR(ccy, t_{i-1}, t_i)  captured in-advance at t_{i-1}
    #   CMS:    rate_i = CMS(ccy, t_{i-1}, swap_n, swap_freq)
    #   OIS:    rate_i = OIS(index, t_{i-1}, t_i)  compounded in arrears
    float_cf_i = (rate_i + spread) * τ_i * notional
    emit Payment(date=t_i, amount = recv_sign*recf_cf_i + pay_sign*pay_cf_i)

```

For an in-advance float index the rate is fixed at the period **start** (t_{i-1}): the expansion emits a Fixing at t_{i-1} storing the rate in a state variable and the Payment at t_i references it (the $t - \tau$ arithmetic pattern). A **cancellable / Bermudan** swap additionally emits Decision events at the call dates with `immediate` = the swap's residual value and the issuer/holder polarity.

B.4 forward

```

forward NAME { underlying, strike, maturity, long=true|false }
sign = (+1 if long else -1)
emit Payment(date=maturity, amount = "sign * (UNDERLYING - strike)")

```

B.5 convertible — the full structure

A `convertible` self-expands (no events block needed). Components, all on the coupon/observation grid generated from `notional`, `maturity`, `coupon`, `frequency`, `ratio`, `[issuer, recovery]`:

```

conv_price = notional / ratio                                # the conversion reference level
risky      = issuer is given

1. Coupon + redemption leg – exactly B.2 (risk-free or survival-weighted by issuer).
2. convert { window=[a,b]; contingent when COND } :
    on each grid date in [a,b] (gated by COND if `contingent`), a holder MAX
    Decision whose immediate value is the conversion payoff:
        convexpr = ratio * UNDERLYING                        # shares received
        payoff   = where(COND, convexpr, 0) if contingent else convexpr
    The holder takes max(convert payoff, continuation value).
3. call { schedule; price; soft when ... for N of last M days; hard from; make_whole } :
    issuer MIN Decision at each call date; immediate = call price (+ make-whole:
    + PV of remaining coupons via DF). `soft` calls are gated on a trailing
    trigger (UNDERLYING ≥ trigger for N of the last M observations); `hard`
    calls are unconditional from a date.
4. put { dates; price } :
    holder MAX Decision at each put date; immediate = put price.
5. reset { dates; floor } :
    on each reset date, lower conv_price toward a floor:
        emit Fixing(do = "conv_price_state = max(conv_price_state, notional/
max(level,floor))")

```

Decisions interleave by `(date, seq)`; a holder-MAX convert and an issuer-MIN call on the same date stay distinct sub-events (the issuer decision typically takes `seq=0`, the holder response follows). `conv_price` is a reserved name inside the convertible's clauses. Make-whole and issuer-credit terms require the rate / credit doors (Appendix A.8–A.9).

Appendix C — Day-count and schedule-roll conventions

These determine accrual fractions τ_i and the period boundaries of swap/bond legs. They change cashflow **amounts**, so an implementation must reproduce them exactly. Inputs are year-fractions from `as_of`; where a `calendar` + `as_of` are supplied, the true calendar-date formulas are used, otherwise the year-fraction approximations below.

C.1 Accrual fraction $\tau(t_a, t_b)$ by day-count

Let `raw` = `t_b` - `t_a` (year-fractions) and `days` = `raw` · 365.25.

Day-count	Formula
ACT/365, ACT/365.25, ACT/ACT	<code>raw</code> (year-fractions are already ACT/365.25-based)
ACT/360, ACTUAL/360	<code>raw</code> · 365.25 / 360
30/360, BOND (no calendar)	<code>raw</code> (fallback to year-fraction when no calendar)
30/360, BOND (with calendar + <code>as_of</code>)	the true 30/360 (Bond Basis) formula on the adjusted calendar dates (C.2)
30E/360 (with calendar + <code>as_of</code>)	the European 30/360 formula (C.2)

Default `day_count` is 30/360. Unknown conventions are an error.

For OIS per-day compounding the same denominators apply at the daily level: $\tau_k = \text{days}_k / 360$ for ACT/360, $\text{days}_k / 365$ for ACT/365, else `raw_k`.

C.2 The 30/360 calendar formula

Given calendar dates `d1` = (Y1,M1,D1), `d2` = (Y2,M2,D2):

```
30/360 (Bond Basis):
    if D1 == 31:          D1 = 30
    if D2 == 31 and D1 == 30: D2 = 30
30E/360 (European):
    if D1 == 31:          D1 = 30
    if D2 == 31:          D2 = 30
 $\tau = ( 360*(Y2 - Y1) + 30*(M2 - M1) + (D2 - D1) ) / 360$ 
```

(The end-of-February end-of-month adjustments of the full ISDA Bond Basis apply when a calendar is supplied; without a calendar the year-fraction fallback is used.)

C.3 Period-boundary generation (stub conventions)

Given `start_t`, `end_t`, and period (the frequency year-fraction), the boundaries `t_0` < `t_1` < ... < `t_n` are generated per stub (default `short_back`):

```
short_back / long_back (step FORWARD from start):
    dates = [start_t]; d = start_t
    while d < end_t - 1e-9:
        d = min(d + period, end_t); dates.append(d)
    if long_back and len(dates) >= 3:
        # merge the final short stub into the previous full period:
        if (dates[-1] - dates[-2]) is a short stub: remove dates[-2]

short_front / long_front (step BACKWARD from end):
    dates = [end_t]; d = end_t
    while d > start_t + 1e-9:
        d = max(d - period, start_t); dates.insert(0, d)
```

```

if long_front and len(dates) >= 3:
    # merge the initial short stub into the following full period:
    if (dates[1] - dates[0]) is a short stub: remove dates[1]

```

A **short** stub is the leftover partial period (shorter than period); a **long** stub merges that leftover into its neighbour so the first/last period is longer than period. `calendar / bda` (business-day adjustment, e.g. `modfollowing`) then roll each boundary to a good business day when a calendar is supplied; without one the raw year-fractions are used directly.

C.4 Frequency codes

Normalised (upper-cased) to ISDA single letters, with year-fractions:

Code	Aliases	Year-fraction
A (annual)	Y, 1Y, 12M, ANNUAL	1.0
S (semi-annual)	6M, SEMIANNUAL	0.5
Q (quarterly)	3M, QUARTERLY	0.25
M (monthly)	1M, MONTHLY	1/12

Tenor units as year-fractions (for date atoms): $D = 1/365.25$, $W = 7/365.25$, $M = 1/12$, $Y = 1.0$.

C.5 Completeness note

With Appendix A (every function body), Appendix B (every instrument expansion), and Appendix C (the accrual/roll conventions), together with the four layers of the Execution Specification and the `SimulatedMarketData` contract of §2, an implementer has a closed specification: no built-in, instrument, or convention is left to guess. The remaining freedom is purely the host language's numeric idioms, which the Engine Specification's cross-language appendix constrains at the points that affect reproducibility (quantile type, standardisation `ddof`, k-means seeding, tie-breaks).

Appendix D — CDO tranche instrument, the closed-form CCR route, and the path-gate guard

D.1 CDO instrument self-expansion

A CDO C { index, attach, detach, spread, schedule, config, upfront, IsUpfrontPayer, nominalExchange } block plus enter C lowers to: (i) an effective date recorded from the enter site (value is zero before it); (ii) a `cdo_instruments` spec entry; and (iii) `CDO_VALUE(index=, attach=, detach=, spread=, schedule=[...], config=, effective=, upfront=, is_upfront_payer=, nominal_exchange=)` wherever the instrument name appears in an expression (receive C). The premium and protection legs are internal to the closed form (tranche_pv_paths); the script writes neither.

```

enter C          -> records C.effective = date_of(enter)          # no cashflow
receive C        -> receive CDO_VALUE(... effective=C.effective ...)

```

D.2 Closed-form CCR route (pricing = closedform)

```
compile_script(... pricing == "closedform" ...):
    for each cdo_instrument: build closure cv_i(smd, dates) = tranche_pv_paths(...
spec_i ...)
    product._cdo_control_variates = [cv_i, ...]                # regression control
    variate
    if closedform requested and NOT path-gated (D.3):
        product._cdo_analytical_pricer = AnalyticalPricer(name, pv_fn =  $\sum_i$  cv_i)

    resolve_analytical_pricer_for_deal(deal) -> product._cdo_analytical_pricer (or None)
    bank_pv_paths: if pricer resolves and not force_regression ->
        return bank_direction * pricer.evaluate(smd_p, dates)    # no
    regression
```

D.3 The path-gate guard (_cdo_closedform_path_gate)

`closedform` is exact only when the CDO value drives the cashflow *bare*. The guard scans the resolved events; with light **taint dataflow over set assignments** it refuses (raising `ValueError("... path-gated ...")`) when a `CDO_VALUE` cashflow is combined with a path-state gate.

```
GATE = {IS_ALIVE, step(, indicator(, heaviside, barrier, knock,
    DEFAULT_TIME, SURVIVAL(, NUM_ALIVE, NUM_DEFAULT)}

# pass 1: collect `set` assignments target = rhs from event `do` strings
# pass 2: taint fixpoint – target tainted if rhs has a GATE token OR references a
tainted var
# pass 3: for each CDO_VALUE cashflow, strip the CDO_VALUE(...) span; flag if the
# remainder holds a GATE token OR references a tainted var
```

This catches inline gating (`receive C * step(IS_ALIVE ...)`) and one routed through a state variable (`set g = step(...); receive C * g, transitively`); a **constant** multiplier is not flagged. Not caught: a CDO value first stored in a variable then gated downstream, and path reads via tokens outside `GATE` — keep `closedform` off for those.

D.4 Verification

`tools/verify_engine_docs.py` reproduces every numeric claim (EL enumeration match, `bypass == bank_direction × closed form with max|diff| = 0`, runoff decay, equality with an independent `tranche_pv_paths` on the DSL-resolved schedule, and the guard refusal); `tools/gen_payoff_catalog.py` / `gen_dsl_constructs_doc.py` parse-verify every signature and construct example. All are gated by the test suite.